

Dokumentation zur Einsendeaufgabe TST-E1: Projektaufgabe Testen

A1 – Unit-Tests & Fehlerbericht

Vorgehen & KI-Nutzung

Für die Aufgabe A1 wurde ein Rechenmodul `Calculator` entworfen. Die KI schlug eine Implementierung für die Methoden `add` und `divide` vor. Der Fokus lag auf der Absicherung von Fehlern mittels Exceptions bei ungültigen mathematischen Operationen (Division durch Null).

KI-Nutzung

Tool: KI-Modus von Google

Korrigieren von Fehlern, Überprüfen vom Code auf Richtigkeit, allgemeine Erklärungen, Erstellen von `CalculatorTest`

Kritik & Korrekturen (Fehlerbericht)

Bei der Umsetzung traten diese Probleme auf:

1. **Paket-Konflikt in der IDE:** Beim Anlegen der Ordner in IntelliJ wurde die Klasse `CalculatorTest` zunächst im falschen Verzeichnis abgelegt, was zu Kompilierungsfehlern führte.

Lösung: das Paket unter `src/test/java` exakt spiegelbildlich zum `main`-Zweig neu legen.

2. **Git-Blockade durch SonarQube beim Push:**

Ein Stolperstein war das SonarQube-Plugin beim Hochladen auf GitHub. Obwohl die lokalen Commits durchgingen, hat das Plugin den eigentlichen Push-Vorgang im Hintergrund blockiert, weil es unzufrieden mit den kosmetischen Test-Warnungen war. Ich stand kurz vor einem Fragezeichen, warum im Browser nichts ankam.

Lösung: Ich musste den Push in IntelliJ erzwingen, indem ich die Warnungen im Pop-up-Fenster explizit mit "Push Anyway" überschrieben habe. Erst danach ging die Meldung `Pushed to origin/master` durch und die TDD-Historie war endlich auf GitHub sichtbar.

3. **Grenzwerte übersehen:** Die KI hat zwar gecheckt, ob der Nenner `b == 0` ist. Mir ist beim Durchgehen aber aufgefallen, dass wir für ein wirklich sicheres System auch negative Zahlen testen müssen. Nur die Zeilen abzuhaken reicht nicht – man muss die Grenzwerte im Auge behalten
4. ![]

The screenshot shows an IDE window titled "projekt-testen" on the "master" branch. The main editor displays the `pom.xml` file for "projekt-testen". The XML content is as follows:

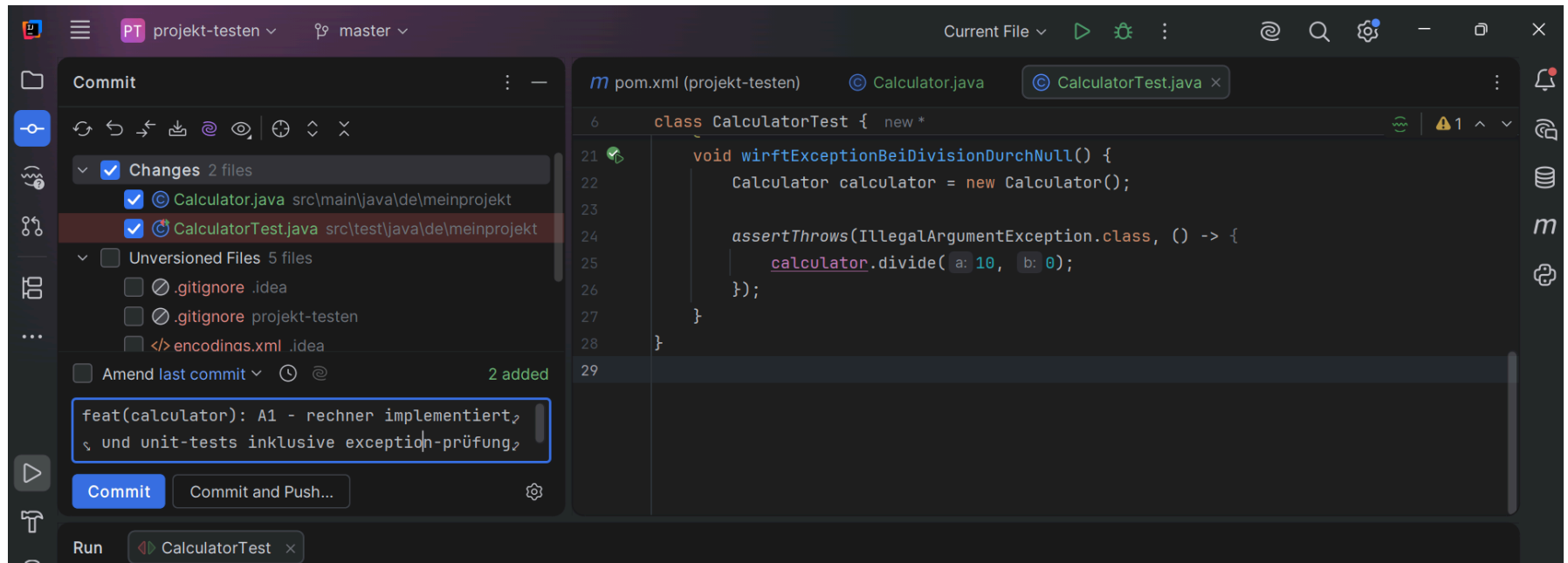
```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>de.meinprojekt</groupId>
  <artifactId>projekt-testen</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>
  <name>projekt-testen</name>
  <description>projekt-testen</description>
  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-jar-plugin</artifactId>
        <version>3.2.0</version>
      </plugin>
    </plugins>
  </build>
</project>
```

The commit dialog on the left shows the commit message: "initialisiere maven projekt mit junit und mockito". The "Commit" button is highlighted. The "Changes" panel on the left lists the files to be committed: `.gitignore`, `encodings.xml`, `misc.xml`, and `vcs.xml`. The "Commit" button is highlighted.

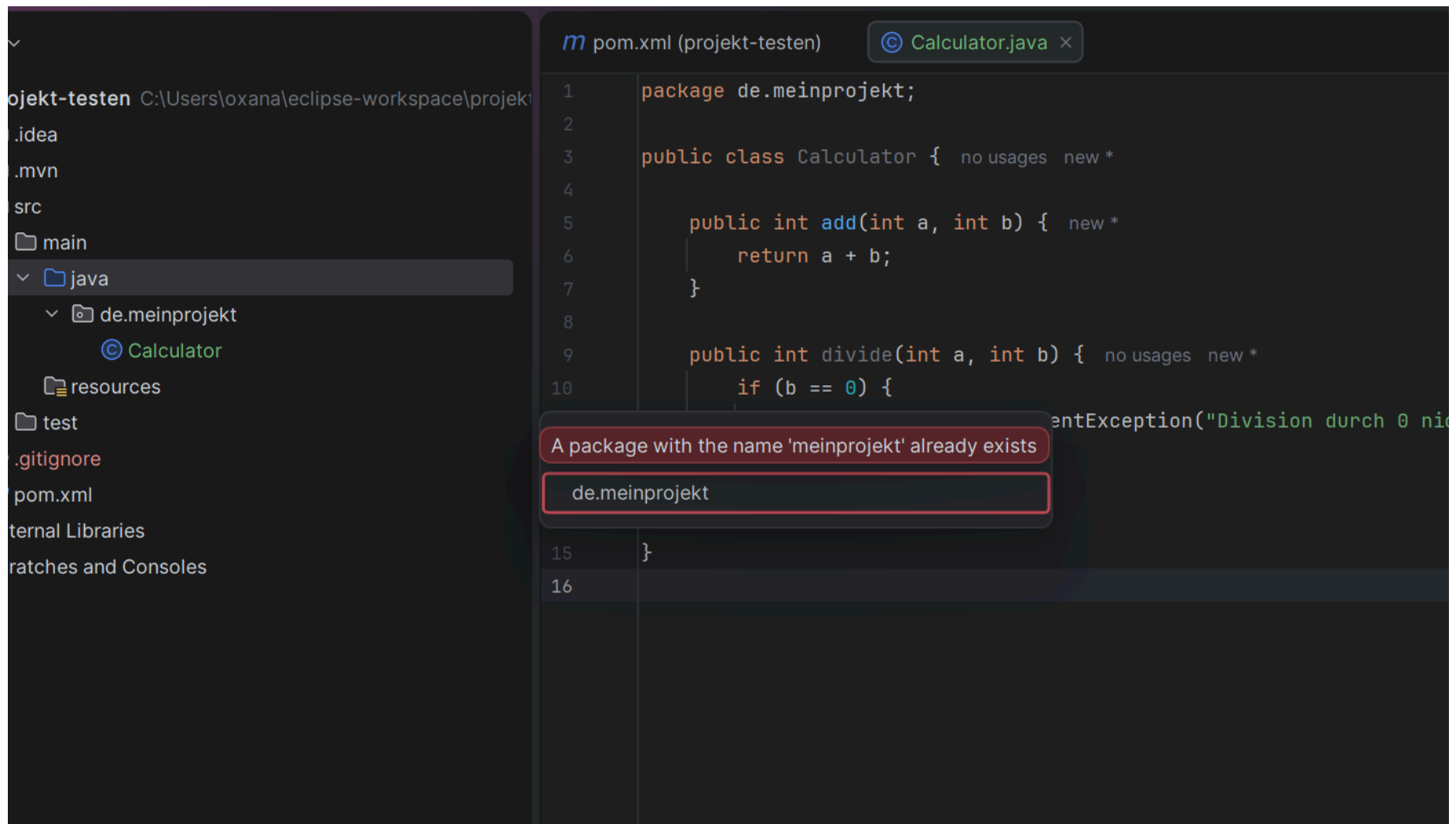
On the right, a notification panel shows the following messages:

- Commit contains problems**
5 errors
[Review code analysis](#) [Ignore](#)
- 1 file committed**
initialisiere maven projekt mit junit und mockito
[Edit Commit Message...](#) [More](#)
- IDE project settings can be added to Git**
[View Files](#) [Always Add](#) [Don't Ask Again](#)

The bottom status bar shows the file is "pom.xml" in the "projekt-testen" directory, with a line length of 75, LF line endings, UTF-8 encoding, and 4 spaces for indentation.



5. Screenshot 2026-06-24 153300.png]]



A2 – TDD Shopping Cart (Vorgehen & Fehlersuche)

Wie ich vorgegangen bin und der TDD-Zyklus

Beim Warenkorb habe ich mich an die TDD-Regeln gehalten. Meine Schritte: Erst den Test schreiben (alles leuchtet rot), dann den minimalen Code bauen, damit der Test besteht (alles wird grün). Am Ende habe ich noch ein paar wichtige Randfälle mit Exceptions abgesichert.

Fehler:

Als ich den Preis und die Menge im `CartItem` absichern wollte, hat es mir plötzlich den kompletten Code zerschossen:

Unten im Build-Fenster ploppte der Fehler `java: Dateiende beim Parsen erreicht (End of file reached)` auf.

Die Ursache: Ich hatte beim Erweitern des Konstruktors geschwungene Klammern durcheinandergebracht. In Zeile 11 hatte ich eine `}` zu viel eingebaut, die den Konstruktor viel zu früh zugemacht hat. Die Variablen-Zuweisungen darunter standen dadurch völlig verloren im Raum.

Lösung: Ich habe den fehlerhaften Klammer-Knoten aufgedröselt, die überflüssige Klammer gelöscht und alles sauber neu strukturiert. Danach sind die JUnit-Tests sind alle grün durchgelaufen.

Gefundener Edge-Case (KI-Nutzung): Die KI fing negative Preise ab. Mir ist dann aber aufgefallen, dass eine Produktmenge von `0` oder weniger im Warenkorb unsinnig ist. Also habe ich die Bedingung `quantity <= 0` miteingebaut und über eine `IllegalArgumentException` abgesichert.

PT projekt-testen master

Current File

ShoppingCartTest.java CartItem.java ShoppingCart.java

Commit

Changes 1 file

CartItem.java src\main\java\de\mein

Unversioned Files 5 files

Amend last commit

test(cart): red - exception bei negativem preis prüfen

Commit Commit and Push...

6 class ShoppingCartTest {

22 void produktKannEntferntWerden() {

23 ShoppingCart cart = new ShoppingCart();

24 cart.addItem(name: "Apfel", price: 2.0, quantity: 2); // Summe wäre 4.0

25

26 cart.removeItem(name: "Apfel");

27

28 assertEquals(expected: 0.0, cart.getTotal());

29 }

30 @Test

31 void negativerPreisWirftException() {

32 ShoppingCart cart = new ShoppingCart();

33

34 assertThrows(IllegalArgumentException.class, () -> {

35 cart.addItem(name: "Apfel", price: -1.0, quantity: 1);

36 });

Run ShoppingCartTest

Shopp 60 ms

pro: 53 ms

prod 2 ms

nega 3 ms

4 tests passed 4 tests total, 60 ms

C:\Users\oxana\.jdk\temurin-23.0.2\bin\java.exe ...

Process finished with exit code 0

Performance

Show Results

00:00

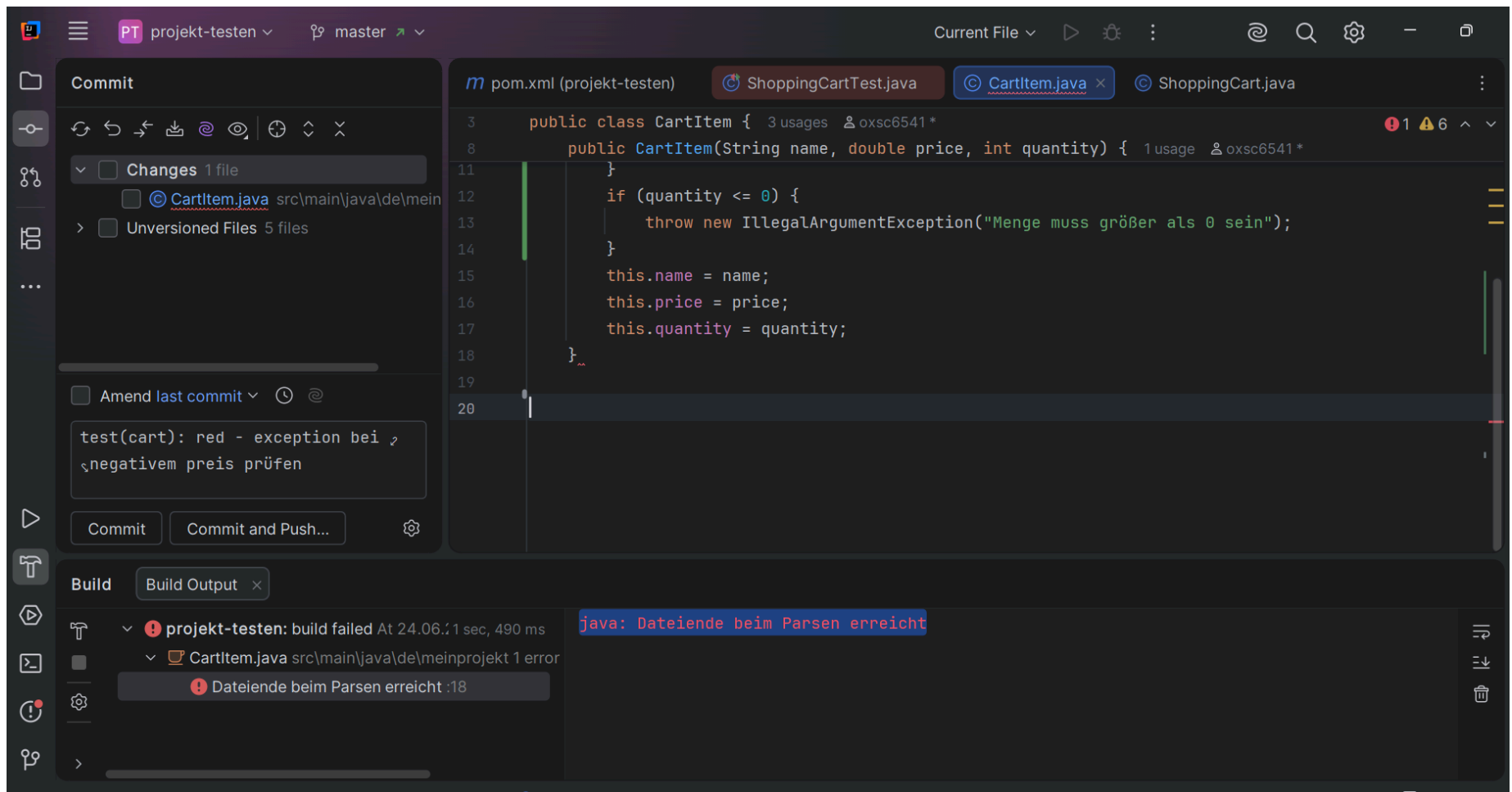
projekt-testen > src > test > java > de > meinprojekt > ShoppingCartTest > negativerPreisWirftException

33:9 CRLF UTF-8 4 spaces

29°C Sonnig

Suche

17:48:36 24.06.2026



A3 – Mocking

Warum genau die Methode gemockt werden musste

Die Methode `getDiscountPercentage` des `DiscountService` wurde herausgemockt, da sie im Betrieb eine externe Web-API über das Internet abfragt. Im Unit-Test würde das zu sehr langsamen Testlaufzeiten führen, wir wären von einer Internetverbindung abhängig und wenn die API mal offline ist, schlagen unsere Tests fehl. Durch das Mocking lässt sich das

Verhalten der API (inklusive fehlerhafter, unmöglicher Rückgabewerte von über 100 %) schnell und kontrolliert simulieren. Im Code kamen die Annotationen wie `@Mock`, `@InjectMocks` sowie `when().thenReturn()` und `verify()` zum Einsatz.

KI-Benutzung: KI-Modus von Google und Claude Sonnet 4.6

Korrigieren von Fehlern, Überprüfen vom Code auf Richtigkeit, allgemeine Erklärungen, Erstellen von `PriceCalculatorTest`

Kritik & Korrekturen (Der Java-23-Absturz)

Bei der Ausführung der Mockito-Tests gab es direkt einen Rückschlag mit einem Absturz:

- **Fehlermeldung:** `Process finished with exit code -1` in einer riesigen Fehlermeldung bezüglich `Byte Buddy`.
- **Ursache:** Da ich mit dem neueren Java 23 (Temurin) arbeite, kam die ältere, von der KI vorgeschlagene Mockito-Version `5.11.0` (bzw. die darin genutzte Bibliothek `Byte Buddy`) ins Schleudern. Java 23 wurde noch nicht unterstützt. Ein erster Versuch, die Version blind in der `pom.xml` hochzuschrauben, scheiterte an einem XML-Syntaxfehler.
- **Die Lösung (KI-Hilfe & Kritik):** Ich habe die Fehlermeldung bei Claude Sonnet 4.6 eingegeben. Die KI schlug mir vor, entweder die Versionen mühsam anzupassen oder ein experimentelles Flag zu setzen. Ich habe mich für den einfachen Weg entschieden und Claudes Lösung in die `pom.xml` eingebaut: Unter dem `maven-surefire-plugin` wurde das Argument `<argLine>-Dnet.bytebuddy.experimental=true</argLine>` ergänzt.
- **Ergebnis:** Nach einem Klick auf den Maven-Reload-Button akzeptiert Java 23 den Byte-Buddy-Agenten nun. Die Tests laufen fehlerfrei mit `exit code 0` durch.

PT

projekt-testen

master

Current File

Updates available. IDE and Project Settings

Project

projekt-testen

idea

.mvn

src

main

java

de.meinprojekt

DiscountService

PriceCalculator

resources

test

target

.gitignore

pom.xml

External Libraries

Run

ShoppingCartTest

Shopp 60 ms

pro 53 ms

prod 2 ms

nega 3 ms

3

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

public class PriceCalculator {

}

public double calculateFinalPrice(double basePrice, String customerType) {

// Holt den Rabatt von der externen Quelle

double discount = discountService.getDiscountPercentage(customerType);

// Kleine Absicherung gegen korrupte API-Daten

if (discount < 0 || discount > 100) {

throw new IllegalStateException("Ungültiger Rabattwert von der API");

}

return basePrice * (1 - (discount / 100.0));

}

}

4 tests passed 4 tests total, 60 ms

C:\Users\oxana\.jdk\temurin-23.0.2\bin\java.exe ...

Process finished with exit code 0

Performance

Show Results

00:00

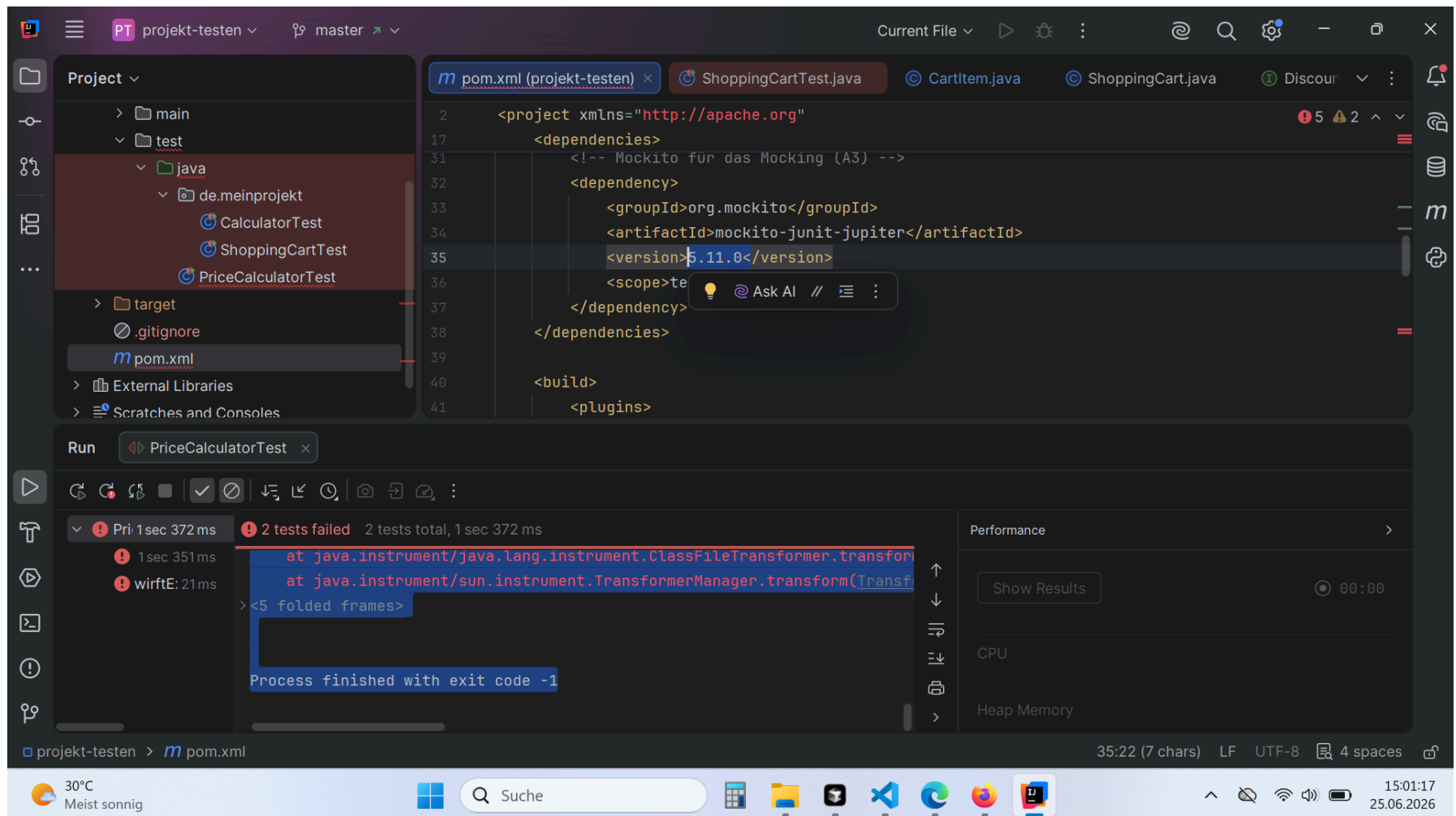
projekt-testen > src > main > java > PriceCalculator

24:1 CRLF UTF-8 4 spaces

30°C Meist sonnig

Suche

14:47:39 25.06.2026



A4 – Mutation Testing (Mein Weg & Auswertung)

Was ich gemacht habe

Zusätzlich zu den Mocks habe ich PIT (Pitest) über mein Projekt laufen lassen, um zu prüfen, wie "hartnäckig" meine Unit-Tests wirklich sind. Das Tool hat 25 Mutanten generiert. 20 davon haben meine Tests sofort gekillt (80 % Mutationsabdeckung). 5 Mutanten haben überlebt. Hier sind zwei davon, die mir besonders aufgefallen sind:

Überlebende Mutanten im Detail:

1. Mutant in `CartItem.java` (Conditionals Boundary Mutator)

- **Was der Mutant gemacht hat:** PIT hat im Konstruktor bei der Prüfung der Menge (`if (quantity <= 0)`) die Bedingung heimlich zu `if (quantity < 0)` abgeändert.
- **Warum er überlebt hat:** Der Test prüft zwar, ob eine ungültige negative Menge abgefangen wird. Ich habe aber vergessen, den Grenzwert `0` zu testen! Bei einer Menge von `0` hätte der mutierte Code den Fehler einfach durchgewinkt, ohne dass mein Test angeschlagen hätte.
- **Wie ich ihn kille:** Ich müsste im `ShoppingCartTest` einen Grenzwertest einbauen, der versucht, ein Item mit exakt der Menge `0` zu erstellen, und erwartet, dass eine `IllegalArgumentException` fliegt.

2. Mutant in `ShoppingCart.java` (Math Mutator / Return Vals)

- **Was der Mutant gemacht hat:** PIT hat in der `getTotal()` -Methode bei der Berechnung der Summe herumgepfuscht / den Rückgabewert verändert.
- **Warum er überlebt hat:** Bei der Berechnung der Artikelsumme (`price * quantity`) schleifen sich durch Rundungsfehler oder ungenaue Assertions manchmal Mutationen durch, wenn die Testwerte (wie `2.0 * 2`) zu "glatt" sind und ein leicht veränderter mathematischer Operator zufällig das gleiche Ergebnis liefert.
- **Wie ich ihn beseitige:** Ich müsste im Test krumme Werte verwenden (z. B. Preis `2.99` , Menge `3`) und mit einem genauen Delta im `assertEquals` prüfen, damit jede noch so kleine Abweichung den Test reißt.

PT projekt-testen master

Current File

51:1 LF UTF-8 4 spaces

Commit

Changes 2 files

CartItem.java src\main\java\de\mein

m pom.xml projekt-testen

Unversioned Files 5 files

Amend last commit

feat(discount): A3 -
mockito-tests laufen
erfolgreich unter java 23
mittels bytebuddy experimental
flag

CommitCommit and Push...

m pom.xml (projekt-testen)

```
2 <project xmlns="http://apache.org"
40 <build>
41 <plugins>
43 <plugin>
44 <groupId>org.apache.maven.plugin
45 <artifactId>maven-surefire-plugi
46 <version>3.2.5</version>
47 <configuration>
48 <argLine>-Dnet.bytebuddy.exp
49 </configuration>
50 </plugin>
51 <!-- PIT für das Mutation Testing (A
52 <plugin>
53 <groupId>org.pitest</groupId>
54 <artifactId>pitest-maven</artifa
55 <version>1.16.1</version>
56 <dependencies>
57 <dependency>
58 <groupId>org.pitest</gro
59 <artifactId>pitest-junit
60 <version>1.2.1</version>
61 </dependency>
62 </dependencies>
63 </configuration>
```

TextDependency Analyzer

Maven

m projekt-testen

Lifecycle

Plugins

clean (org.apache.maven.plugins:maven-clean-plugin

compiler (org.apache.maven.plugins:maven-compiler

deploy (org.apache.maven.plugins:maven-deploy-plu

install (org.apache.maven.plugins:maven-install-plugi

jar (org.apache.maven.plugins:maven-jar-plugin:3.4.1

pitest (org.pitest:pitest-maven:1.16.1)

pitest:help

pitest:mutationCoverage

pitest:report

pitest:report-aggregate

pitest:report-aggregate-module

pitest:scmMutationCoverage

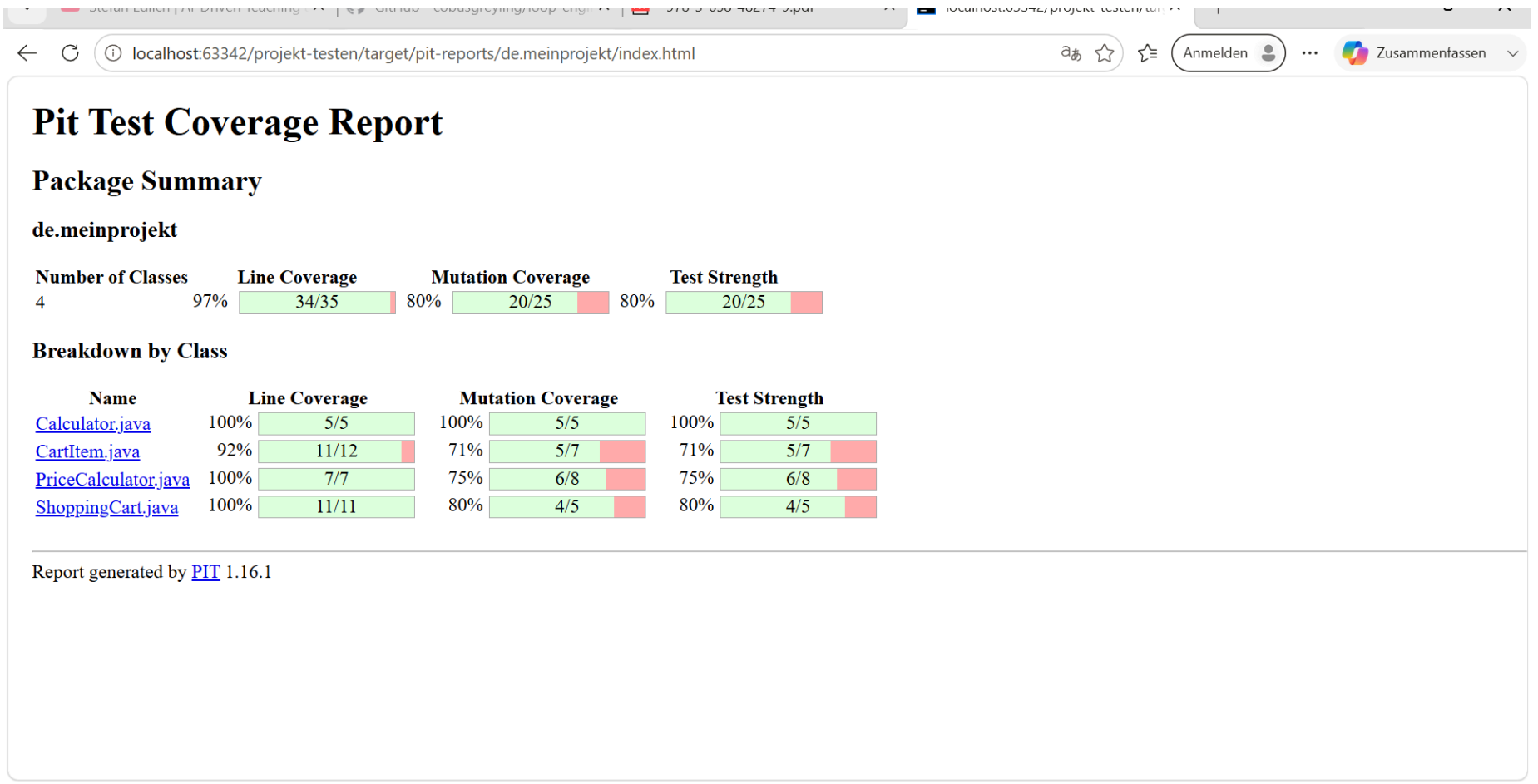
resources (org.apache.maven.plugins:maven-resourc

site (org.apache.maven.plugins:maven-site-plugin:3.1

surefire (org.apache.maven.plugins:maven-surefire-p

Dependencies

Repositories



Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
4	97% 34/35	80% 20/25	80% 20/25

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
de.meinprojekt	4	97% 34/35	80% 20/25	80% 20/25

Report generated by [PIT](#) 1.16.1

Enhanced functionality available at [arcmutate.com](#)